



**Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026**

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251233
Nama Lengkap	Mikael Gratianus Satrio Adi Kuncoro
Minggu ke / Materi	07 / Percabangan dan Perulangan Kompleks

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026**

Struktur Percabangan Kompleks

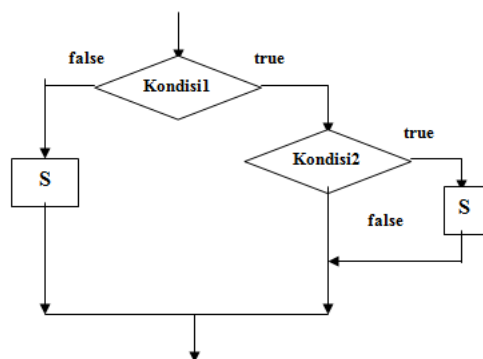
Percabangan di mana kondisi pemilihan tidak hanya satu, tetapi dapat terdiri atas banyak alternatif, perintah-perintah yang dikerjakan juga bisa lebih dari satu. Ada banyak bentuk dalam percabangan kompleks yaitu:

- Percabangan kompleks bentuk 1:

```
1 > if kondisi1:  
2 >     if kondisi2:  
3 >         S  
4 >         S  
5 > else:  
6 >     S  
7 >     S
```

Gambar 1.1: Struktur percabangan kompleks bentuk 1

Berikut merupakan gambar flowchart nya:



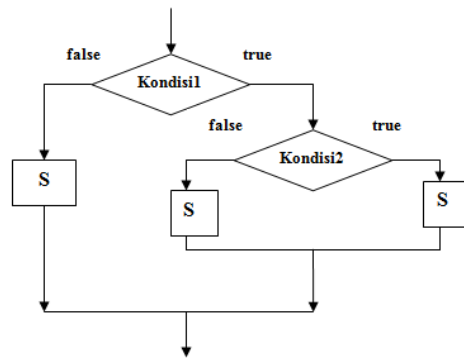
Gambar 1.2: Flowchart percabangan kompleks bentuk 1

- Percabangan kompleks bentuk 2:

```
1 > if kondisi1:  
2 >     if kondisi2:  
3 >         S  
4 >         S  
5 >     else:  
6 >         S  
7 >         S  
8 > else:  
9 >     S  
10 >    S
```

Gambar 1.3: Struktur percabangan kompleks bentuk 2

Berikut merupakan gambar flowchart nya:



Gambar 1.3: Flowchart percabangan kompleks bentuk 2

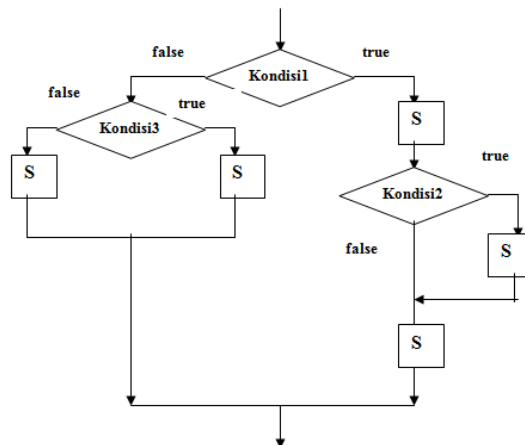
- Percabangan kompleks bentuk 3:

```

1  if kondisi1:
2      S
3  if kondisi2:
4      S
5      S
6  else:
7      if kondisi3:
8          S
9          S
10 else:
11     S
12     S
  
```

Gambar 1.4: Struktur percabangan kompleks bentuk 3

Berikut merupakan gambar flowchart nya:



Gambar 1.5: Flowchart percabangan kompleks bentuk 3

- Percabangan kompleks bentuk 4:

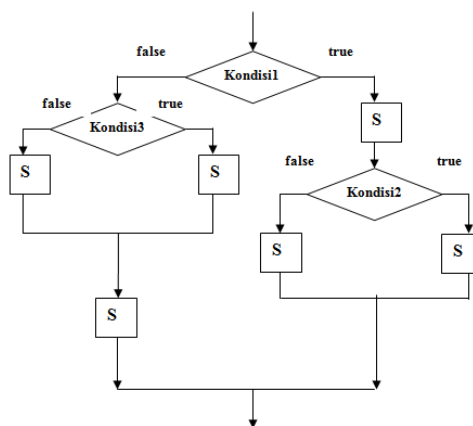
```

1- if kondisi1:
2-     S
3-     if kondisi2:
4-         S
5-         S
6-     else:
7-         S
8-         S
9- else:
10-    if kondisi3:
11-        S
12-        S
13-    else:
14-        S
15-        S
16 S

```

Gambar 1.6: Struktur percabangan kompleks bentuk 4

Berikut merupakan gambar flowchart nya:



Gambar 1.7: Flowchart percabangan kompleks bentuk 4

- Percabangan kompleks bentuk 5:

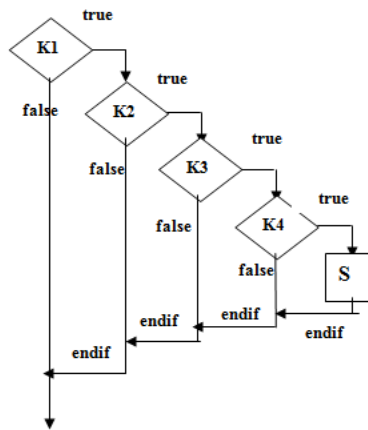
```

1- if kondisi1:
2-     if kondisi2:
3-         if kondisi3:
4-             if kondisi4:
5-                 S

```

Gambar 1.8: Struktur percabangan kompleks bentuk 5

Berikut merupakan gambar flowchart nya:



Gambar 1.9: Flowchart percabangan kompleks bentuk 5

- Percabangan kompleks bentuk 6:

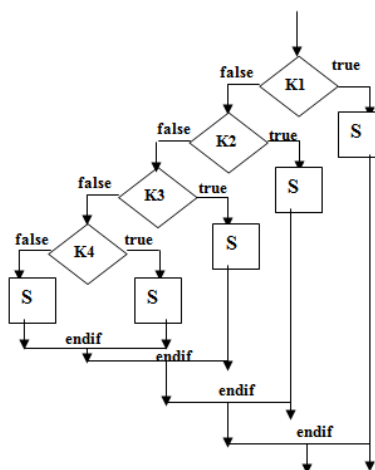
```

1- if kondisi1:
2-     S
3- else:
4-     if kondisi2:
5-         S
6-     else:
7-         if kondisi3:
8-             S
9-         else:
10-            if kondisi4:
11-                S
12-            else:
13-                S

```

Gambar 2.1: Struktur percabangan kompleks bentuk 6

Berikut merupakan gambar flowchart nya:



Gambar 2.2: Flowchart percabangan kompleks bentuk 6

Pada kasus tertentu **IF bertingkat (nested if)** itu lebih efisien karena program tidak perlu mengecek semua kondisi, karena program hanya akan lanjut mengecek kondisi berikutnya kalau kondisi sebelumnya terpenuhi.

Sedangkan pada **IF biasa (banyak if terpisah)**, semua kondisi akan tetap dicek satu per satu, walaupun sebenarnya sudah ada kondisi yang benar, jadi prosesnya akan lebih lama. Berikut contoh strukturnya:

Nested if	If biasa
<pre> 1 if (kondisi1): 2 instruksi1 3 elif(kondisi2): 4 instruksi2 5 elif(kondisi3): 6 instruksi3 7 elif(kondisi4): 8 instruksi4 </pre>	<pre> 1 if (kondisi1): 2 instruksi1 3 if(kondisi2): 4 instruksi2 5 if(kondisi3): 6 instruksi3 7 if(kondisi4): 8 instruksi4 </pre>

Gambar 2.3: Contoh struktur nested if dan if biasa.

Struktur Perulangan Kompleks

- Break

Perintah ini digunakan untuk menghentikan perulangan saat kondisi tertentu terpenuhi, yang diimplementasikan menggunakan perintah IF. Berikut source codenya:

<pre> 1 for i in range(1000): 2 print(i, end="") 3 if i == 10: 4 break </pre>	Output 012345678910
---	--

Gambar 2.4: Contoh source code dan outputnya.

Program diatas hanya menampilkan angka 1–10 karena perulangan dihentikan oleh break saat i = 10, dan angka 10 tetap tercetak karena perintah print dijalankan sebelum break. Berikut contoh program break lainnya:

<pre> 1 for i in range(1000): 2 if i == 10: 3 break 4 print(i, end="") </pre>	Output 0123456789
---	--

Gambar 2.5: Contoh source code dan outputnya.

Program hanya menampilkan angka 1–9 karena perulangan dihentikan oleh break saat i = 10, dan angka 10 tidak tercetak karena perintah print berada setelah break.

- Continue

Perintah `continue` menyebabkan proses perulangan kembali ke awal mula, dengan mengabaikan statement-statement berikutnya setelah `continue`. Biasanya perintah `continue` juga diimplementasikan menggunakan perintah `IF`. Berikut contoh source codenya:

<pre> 1 for i in range(10): 2 if i == 5: 3 continue 4 print(i, end="") </pre>	Output 012346789
---	---------------------------------------

Gambar 2.6: Contoh source code dan outputnya.

Program di atas tidak menampilkan angka 5 karena pada saat angka 5 akan ditampilkan, perintah `continue` dijalankan, sehingga perintah mencetak di bagian bawahnya tidak akan dikerjakan dan langsung dilanjutkan ke perulangan berikutnya!

- Perulangan Bertingkat

Struktur perulangan kompleks adalah loop bertingkat (loop di dalam loop) yang digunakan untuk masalah berbentuk grid atau matriks seperti array 2D, game board, dan pengolahan citra. Berikut contoh perulangan bertingkat atau nested loop sederhananya:

<pre> 1 n = 5 2 3 for i in range(1, n+1): 4 for j in range(n): 5 print("*", end=" ") 6 print() </pre>	Output <pre> * </pre>
---	---

Gambar 2.7: Contoh source code dan outputnya.

Kode ini mencetak pola bintang 5x5 menggunakan perulangan bertingkat, di mana loop luar(`for i in range(1, n+1)`) untuk baris dan loop dalam(`for j in range(n)`) untuk kolom.

Berikut adalah contoh lainnya:

```

1 while(i<=m):
2     while(j<=n):
3         <lakukan perintah ini di inner>
4         <lakukan perintah itu di inner>
5     <lakukan perintah lain di outer>
6     <lakukan perintah lain lagi di outer>

```

Gambar 2.8: Contoh struktur jika menggunakan `while`.

Perulangan **while i** sebagai outer loop akan menjalankan **while j** sebagai inner loop sebanyak **n (kali)** untuk setiap **m (kali)** perulangan outer loop.

LINK GITHUB: <https://github.com/Rio-code-07/71251233-Rio-PrakAlpro.git>

SOAL 1



```
soal01.py > ...
1  n = int(input("Masukan Bilangan: "))
2
3  for i in range(n-1, 0, -1):
4      prima = True
5      for j in range(2, i-1):
6          if i % j == 0:
7              prima = False
8
9      if prima == True:
10         print("Maka prima terdekat <", n, "adalah", i)
11         break

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu07\soal01.py"
Masukan Bilangan: 12
Maka prima terdekat < 12 adalah 11
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu07\soal01.py"
Masukan Bilangan: 21
Maka prima terdekat < 21 adalah 19
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07> 
```

Gambar 2.9: Source code soal 1.

Penjelasan:

Ini adalah program untuk mencari bilangan prima terdekat dari suatu bilangan yang diinputkan oleh pengguna. Program ini diawali dengan membuat variabel **n** untuk menampung input dari pengguna, lalu membuat perulangan **for i in range(n-1, 0, -1):** yang berarti loop dari **n-1** turun sampai 1, untuk mencari kandidat bilangan prima. Setelah itu membuat variabel penanda yaitu **prima = True** untuk menganggap **i** adalah bilangan prima dulu sebelum di cek. Lalu membuat perulangan **for j in range(2, i-1):** yang diikuti dengan kondisi **if i % j == 0:**, untuk mengecek **i** dengan membaginya dari angka 2 sampai **i-1**, kalau **i** habis dibagi **j**, berarti **i** bukan prima. Lalu menambahkan variabel **prima = False** untuk mengubah variable prima menjadi **False** karena terbukti jika bukan prima. Lalu diluar perulangan **j** ditambahkan kondisi **if prima == True:** untuk mengecek apakah **i** masih prima kalau memang prima maka cetak hasilnya dengan **print("Maka prima terdekat <", n, "adalah", i)** dan terakhir ditambahkan **break** untuk menghentikan loop karena sudah ketemu bilangan prima terdekat nya.

SOAL 2



```
soal02.py > ...
1 import math
2 n = int(input("Masukan n: "))
3
4 for i in range(n, 0, -1):
5     print(math.factorial(i), end=" ")
6     for j in range(i, 0, -1):
7         print(j, end=" ")
8     print()
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

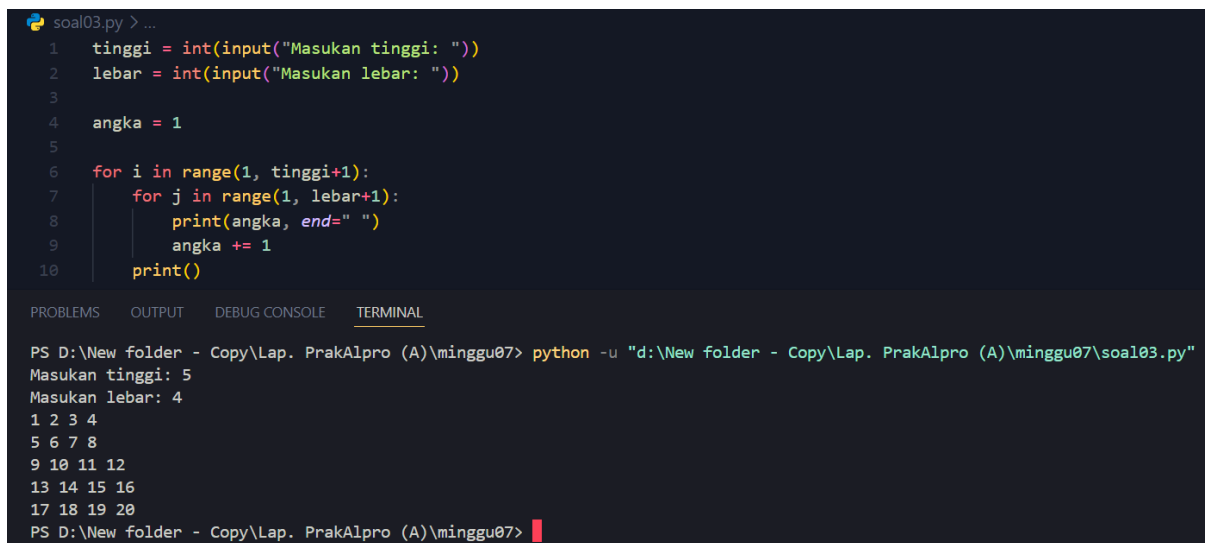
```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu07\soal02.py"
Masukan n: 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07>
```

Gambar 3.0: Source code soal 2.

Penjelasan:

Ini adalah program untuk menentukan deret yang diinputkan secara dinamis oleh pengguna. Program ini diawali dengan mengimport library bawaan yaitu **math** untuk menggunakan fungsi faktorial nantinya. Setelah itu membuat variabel **n** untuk menyimpan imputan dari pengguna. Lanjut membuat perulangan yaitu **for i in range(n, 0, -1):** loop ini turun dari **n** sampai **1** untuk setiap baris. Setelah itu mencetak faktorial dari **i** di awal setiap baris dengan **print(math.factorial(i), end=" ")**. **end=" "**, digunakan agar tidak pindah baris. Setelah itu membuat perulangan lagi untuk mencetak deret angka di setiap baris dengan **for j in range(i, 0, -1):** loop ini turun dari **i** sampai **1**. Setelah itu barulah mencetak angka di perulangan **J**, dan juga mencetak baris kosong untuk memindahkan baris setelah setiap baris selesai dicetak.

SOAL3



```
soal03.py > ...
1  tinggi = int(input("Masukan tinggi: "))
2  lebar = int(input("Masukan lebar: "))
3
4  angka = 1
5
6  for i in range(1, tinggi+1):
7      for j in range(1, lebar+1):
8          print(angka, end=" ")
9          angka += 1
10     print()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07> python -u "d:\New folder - Copy\Lap. PrakAlpro (A)\minggu07\soal03.py"
Masukan tinggi: 5
Masukan lebar: 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
PS D:\New folder - Copy\Lap. PrakAlpro (A)\minggu07>
```

Gambar 3.1: Source code soal 3.

Penjelasan:

Ini adalah program untuk menentukan deret yang diinputkan secara dinamis oleh pengguna. Program diawali dengan membuat variabel **tinggi** dan **lebar** untuk menyimpan inputan dari pengguna dan membuat variabel penampung yaitu **angka** dengan nilai awal yaitu 1, ini untuk menyimpan angka yang akan dicetak. Setelah itu lanjut membuat perulangan yaitu **for i in range(1, tinggi+1):** untuk melooping sebanyak **tinggi** (kali) untuk setiap baris, dan perulangan **for j in range(1, lebar+1):** untuk melooping sebanyak **lebar** (kali) untuk setiap kolom. Lanjut dengan **print(angka, end=" ")** untuk mencetak variabel penampung yaitu **angka** tadi dan **end=" "** agar tidak pindah baris. Setelah itu menambahkan **angka += 1** untuk menambahkan nilai pada variabel penampung yaitu **angka** sebesar **1** setiap langkah. Terakhir mencetak baris kosong untuk memindahkan baris setelah setiap baris selesai di cetak dengan menambahkan fungsi **print()** pada luar perulangan **j** tapi masih didalam perulangan **i**.

